

Querying a Database in a Flask Application : Code-Along (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-application>  <https://github.com/learn-co-curriculum/python-p4-v2-flask-sqlalchemy-application/issues/new>

Learning Goals

- Implement a Flask application to query the database

Introduction

The moment we've all been waiting for has arrived! Let's use our newly-seeded database to put pet information onto the internet!

Setup

This lesson is a code-along, so fork and clone the repo.

Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
```

```
$ export FLASK_RUN_PORT=5555
```

Execute the command `tree` within the `server` directory.

```
$ tree
```

```
.
├── app.py
├── migrations
│   ├── README
│   ├── alembic.ini
│   ├── env.py
│   ├── script.py.mako
│   └── versions
│       └── 51b06098cc9e_initial_migration.py
├── models.py
├── seed.py
├── testing
│   └── codegrade_test.py
```

The commands `flask db init` and `flask db migrate` have already been run, so the `server` directory contains the `migrations` directory, and the directory `server/migrations/versions` contains an initial migration script.

Run the following command to create the `instance` directory with the database and initialize the database from the existing migration script:

```
$ flask db upgrade head
```

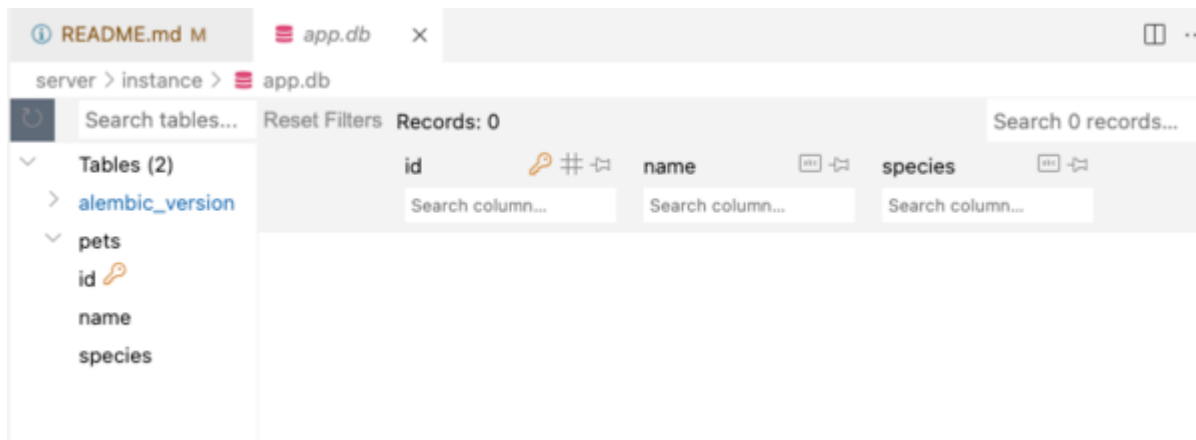
The `instance` folder should now appear along with the database file `app.db` inside it:

```
.
├── app.py
└── instance
```

```
|   └─ app.db
└─ migrations
|   └─ README
|   └─ alembic.ini
|   └─ env.py
|   └─ script.py.mako
|   └─ versions
|       └─ 51b06098cc9e_initial_migration.py
└─ models.py
└─ seed.py
└─ testing
    └─ codegrade_test.py
```

Confirm the database contains an empty `pets` table, either by using the Flask shell or by using a VS Code extension to view the table contents.

```
$ flask shell
>>> Pet.query.all()
[]
>>>
```



Let's seed the table with sample data. Type the following within the `server` directory:

```
$ python seed.py
```

Use the Flask shell to confirm 10 random pets have been added to the database:

```
$ flask shell
>>> Pet.query.all()
[<Pet 1, Robin, Hamster>, <Pet 2, Gwendolyn, Dog>, <Pet 3, Michael, Turtle>, <Pet 4, Austin, Cat>, <Pet 5, Jennifer,
>>>
```

Serving Database Records in Flask Applications

Open `server/app.py` and modify it to add the `index()` view as shown below:

```
# server/app.py
#!/usr/bin/env python3

from flask import Flask, make_response
from flask_migrate import Migrate

from models import db, Pet

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

migrate = Migrate(app, db)

db.init_app(app)
```

```
@app.route('/')
def index():
    response = make_response(
        '<h1>Welcome to the pet directory!</h1>',
        200
    )
    return response

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

We've seen all of this code before in one way or another, but let's take a moment to review:

- Our `app.config` is set up to point to our existing database, and `'SQLALCHEMY_TRACK_MODIFICATIONS'` is set to `False` to avoid building up too much unhelpful data in memory when our application is running.
- Our `migrate` instance configures the application and models for Flask-Migrate.
- `db.init_app` connects our database to our application before it runs.
- `@app.route` determines which resources are available at which URLs and saves them to the application's URL map.
- Responses are what we return to the client after a request and `make_response` helps us with that. It is a function that allows you to create an HTTP response object that you can customize before returning it to the client. It's a useful tool for building more complex responses, especially when you need to set custom headers, cookies, or other response attributes. The included response has a status code of 200, which means that the resource exists and is accessible at the provided URL.

Enter the `server/` directory if you're not there already and type:

```
python app.py
```

In a browser, navigate to `127.0.0.1:5555`. You should see this message in your browser:



Welcome to the pet directory!

Creating Our Views

Let's start working on displaying data in our Flask application.

We'll start by adding another view for pets, searched by ID. Add the new view after the `index()` view:

```
# server/app.py

from flask import Flask, make_response
from flask_migrate import Migrate

from models import db, Pet

# add this view after index()

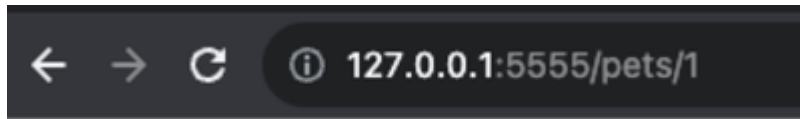
@app.route('/pets/<int:id>')
def pet_by_id(id):
    pet = Pet.query.filter(Pet.id == id).first()
    response_body = f'<p>{pet.name} {pet.species}</p>'

    response = make_response(response_body, 200)

    return response

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Run the application again and navigate to 127.0.0.1:5555/pets/1. Because we generated data with Faker, you will probably not see the same name and species, but your format should match below:



Robin Hamster

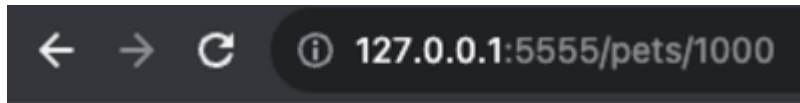
Navigate now to `127.0.0.1:5555/pets/1000`. You will see an error message that suggests something went wrong on your server. We're not tracking 1000 pets right now, but we still don't want our users to see error pages like this. Let's make another small change to the `pet_by_id()` view to fix this:

```
@app.route('/pets/<int:id>')
def pet_by_id(id):
    pet = Pet.query.filter(Pet.id == id).first()

    if pet:
        response_body = f'<p>{pet.name} {pet.species}</p>'
        response_status = 200
    else:
        response_body = f'<p>Pet {id} not found</p>'
        response_status = 404

    response = make_response(response_body, response_status)
    return response
```

404 is the status code for "Not Found". It is generally used for the case we see here: an ID or username that went into the URL but does not exist. Our 404 page is quite minimal, but applications like Twitter and Facebook format them with descriptive messages and the same style and formatting of their website as a whole.



Pet 1000 not found

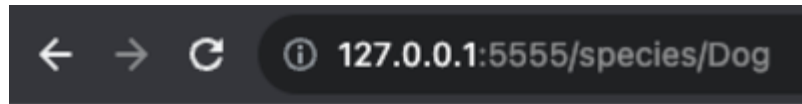
Let's add another view to get all pets for a given species. We'll filter to get all rows that match the `species` route parameter, then loop through the query result to generate response information for each pet.

```
@app.route('/species/<string:species>')
def pet_by_species(species):
    pets = Pet.query.filter_by(species=species).all()

    size = len(pets) # all() returns a list so we can get length
    response_body = f'<h2>There are {size} {species}s</h2>'
    for pet in pets:
        response_body += f'<p>{pet.name}</p>'
    response = make_response(response_body, 200)
    return response
```

The expression `species=species` passed into the `filter_by` function may be a bit confusing. The `species` before the equal sign refers to the table column, while `species` after the equal sign refers to the route parameter.

Let's test this new route (your result will differ). Navigate to `127.0.0.1:5555/species/Dog`.



There are 3 Dogs

Gwendolyn

Jennifer

Jenna

Conclusion

You should now be able to use Flask-SQLAlchemy and Flask-Migrate to send and receive information between databases, web servers, and clients far away. These powerful tools require practice to get used to- our next lesson will give you the chance to write a full-stack Flask application on your own.

If your application for this lesson still isn't working, don't worry! Refer to the solution code below and meet with your peers, instructors, and coaches to iron out any remaining wrinkles.

Solution Code

```
# server/app.py
#!/usr/bin/env python3

from flask import Flask, make_response
from flask_migrate import Migrate

from models import db, Pet
```

```
app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

migrate = Migrate(app, db)

db.init_app(app)

@app.route('/')
def index():
    response = make_response(
        '<h1>Welcome to the pet directory!</h1>',
        200
    )
    return response

@app.route('/pets/<int:id>')
def pet_by_id(id):
    pet = Pet.query.filter(Pet.id == id).first()

    if pet:
        response_body = f'<p>{pet.name} {pet.species}</p>'
        response_status = 200
    else:
        response_body = f'<p>Pet {id} not found</p>'
        response_status = 404

    response = make_response(response_body, response_status)
    return response
```

```
@app.route('/species/<string:species>')
def pet_by_species(species):
    pets = Pet.query.filter_by(species=species).all()

    size = len(pets) # all() returns a list so we can get length
    response_body = f'<h2>There are {size} {species}s</h2>'
    for pet in pets:
        response_body += f'<p>{pet.name}</p>'
    response = make_response(response_body, 200)
    return response

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- [Quickstart - Flask-SQLAlchemy](https://flask-sqlalchemy.palletsprojects.com/en/2.x/quickstart/)  (https://flask-sqlalchemy.palletsprojects.com/en/2.x/quickstart/)
- [Flask-Migrate](https://flask-migrate.readthedocs.io/en/latest/)  (https://flask-migrate.readthedocs.io/en/latest/)
- [Flask Extensions, Plug-ins, and Related Libraries - Full Stack Python](https://www.fullstackpython.com/flask-extensions-plugin-ins-related-libraries.html)  (https://www.fullstackpython.com/flask-extensions-plugin-ins-related-libraries.html)

This tool needs to be loaded in a new browser window

Load Querying a Database in a Flask Application : Code-Along (CodeGrade) in a new window